



Batch 21 — Production + Menu Engineering Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:23

Batch 21 — Production + Menu Engineering Pack

Meat Lovers CIMS powered by YohPal

21A.

database/migrations/037_recipes.sql

```
CREATE TABLE recipes (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  menu_product_id BIGINT NOT NULL,  
  recipe_name VARCHAR(255) NOT NULL,  
  serving_size DECIMAL(12,2) DEFAULT 1,  
  
  is_active BOOLEAN DEFAULT TRUE,  
  
  FOREIGN KEY (menu_product_id) REFERENCES products(id),  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

21B.

database/migrations/038_recipe_items.sql

```
CREATE TABLE recipe_items (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  recipe_id BIGINT NOT NULL,  
  ingredient_product_id BIGINT NOT NULL,  
  
  quantity_required DECIMAL(12,2) NOT NULL,  
  unit_cost DECIMAL(12,2) DEFAULT 0,  
  
  FOREIGN KEY (recipe_id) REFERENCES recipes(id),  
  FOREIGN KEY (ingredient_product_id) REFERENCES products(id),  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

21C.

database/migrations/039_kitchen_production_plans.sql

```
CREATE TABLE kitchen_production_plans (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  production_date DATE NOT NULL,  
  menu_product_id BIGINT NOT NULL,  
  
  planned_quantity DECIMAL(12,2) NOT NULL,  
  produced_quantity DECIMAL(12,2) DEFAULT 0,  
  sold_quantity DECIMAL(12,2) DEFAULT 0,  
  wasted_quantity DECIMAL(12,2) DEFAULT 0,  
  
  production_status ENUM(  
    'PLANNED',  
    'IN_PROGRESS',  
    'COMPLETED',  
    'CLOSED'  
  ) DEFAULT 'PLANNED',
```

```

    created_by BIGINT,

    FOREIGN KEY (menu_product_id) REFERENCES products(id),
    FOREIGN KEY (created_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

21D.

database/migrations/040_ingredient_consumption.sql

```

CREATE TABLE ingredient_consumption (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    production_plan_id BIGINT,
    order_id BIGINT,
    product_id BIGINT NOT NULL,

    consumed_quantity DECIMAL(12,2) NOT NULL,
    consumption_source ENUM(
        'PRODUCTION_PLAN',
        'ORDER',
        'MANUAL_ADJUSTMENT'
    ) NOT NULL,

    consumed_by BIGINT,

    FOREIGN KEY (production_plan_id) REFERENCES kitchen_production_plans(id),
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (consumed_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

21E.

database/migrations/041_food_wastage.sql

```

CREATE TABLE food_wastage (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    product_id BIGINT NOT NULL,
    production_plan_id BIGINT,

    wasted_quantity DECIMAL(12,2) NOT NULL,
    estimated_cost DECIMAL(12,2) DEFAULT 0,

    wastage_reason TEXT,
    declared_by BIGINT,

    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (production_plan_id) REFERENCES kitchen_production_plans(id),
    FOREIGN KEY (declared_by) REFERENCES users(id),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

```

21F. Update

backend/routes/api.php

```

$router->get('/api/production/recipes', [KitchenController::class, 'recipes']);
$router->post('/api/production/recipes', [KitchenController::class, 'createRecipe']);

$router->get('/api/production/plans', [KitchenController::class, 'productionPlans']);
$router->post('/api/production/plans', [KitchenController::class, 'createProductionPlan']);
$router->post('/api/production/plans/{id}/start', [KitchenController::class, 'startProductionPlan']);
$router->post('/api/production/plans/{id}/complete', [KitchenController::class, 'completeProductionPlan']);

$router->post('/api/production/ingredient-consumption', [KitchenController::class,
'recordIngredientConsumption']);
$router->post('/api/production/wastage', [KitchenController::class, 'recordWastage']);

$router->get('/api/production/menu-engineering-report', [KitchenController::class, 'menuEngineeringReport']);
$router->get('/api/production/food-cost-report', [KitchenController::class, 'foodCostReport']);

```

21G. Update

backend/app/Controllers/KitchenController.php

Add methods inside the class:

```
public function recipes(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);

    $stmt = Database::connection()->query(
        "SELECT
            r.*,
            p.product_name AS menu_product_name
        FROM recipes r
        JOIN products p ON p.id = r.menu_product_id
        ORDER BY r.created_at DESC"
    );

    Response::success(['recipes' => $stmt->fetchAll()]);
}

public function createRecipe(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);
    $data = Request::body();
    $db = Database::connection();

    $db->beginTransaction();

    $stmt = $db->prepare(
        'INSERT INTO recipes
        (menu_product_id, recipe_name, serving_size, is_active)
        VALUES (:menu_product_id, :recipe_name, :serving_size, 1)'
    );

    $stmt->execute([
        'menu_product_id' => $data['menu_product_id'],
        'recipe_name' => $data['recipe_name'],
        'serving_size' => $data['serving_size'] ?? 1,
    ]);

    $recipeId = (int) $db->lastInsertId();

    foreach ($data['items'] as $item) {
        $itemStmt = $db->prepare(
            'INSERT INTO recipe_items
            (recipe_id, ingredient_product_id, quantity_required, unit_cost)
            VALUES (:recipe_id, :ingredient_product_id, :quantity_required, :unit_cost)'
        );

        $itemStmt->execute([
            'recipe_id' => $recipeId,
            'ingredient_product_id' => $item['ingredient_product_id'],
            'quantity_required' => $item['quantity_required'],
            'unit_cost' => $item['unit_cost'] ?? 0,
        ]);
    }

    AuditLogger::log(
        (int) $user['id'],
        'PRODUCTION',
        'CREATE_RECIPE',
        $recipeId,
        null,
        $data
    );

    $db->commit();

    Response::success(['recipe_id' => $recipeId, 'Recipe created']);
}

public function productionPlans(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);

    $stmt = Database::connection()->query(
        "SELECT
            kpp.*,
            p.product_name,
            u.full_name AS created_by_name
        FROM kitchen_production_plans kpp
        JOIN products p ON p.id = kpp.menu_product_id"
    );
}
```

```

        LEFT JOIN users u ON u.id = kpp.created_by
        ORDER BY kpp.production_date DESC"
    );

    Response::success(['production_plans' => $stmt->fetchAll()]);
}

public function createProductionPlan(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO kitchen_production_plans
        (production_date, menu_product_id, planned_quantity, created_by)
        VALUES (:production_date, :menu_product_id, :planned_quantity, :created_by)'
    );

    $stmt->execute([
        'production_date' => $data['production_date'],
        'menu_product_id' => $data['menu_product_id'],
        'planned_quantity' => $data['planned_quantity'],
        'created_by' => $user['id'],
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'PRODUCTION',
        'CREATE_PRODUCTION_PLAN',
        null,
        null,
        $data
    );

    Response::success([], 'Production plan created');
}

public function startProductionPlan(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);

    $stmt = Database::connection()->prepare(
        "UPDATE kitchen_production_plans
        SET production_status = 'IN_PROGRESS'
        WHERE id = :id"
    );

    $stmt->execute(['id' => $id]);

    AuditLogger::log((int) $user['id'], 'PRODUCTION', 'START_PRODUCTION_PLAN', $id);

    Response::success([], 'Production plan started');
}

public function completeProductionPlan(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        "UPDATE kitchen_production_plans
        SET production_status = 'COMPLETED',
            produced_quantity = :produced_quantity
        WHERE id = :id"
    );

    $stmt->execute([
        'produced_quantity' => $data['produced_quantity'],
        'id' => $id,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'PRODUCTION',
        'COMPLETE_PRODUCTION_PLAN',
        $id,
        null,
        $data
    );

    Response::success([], 'Production plan completed');
}

public function recordIngredientConsumption(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);
    $data = Request::body();

```

```

$db = Database::connection();

$db->beginTransaction();

$stmt = $db->prepare(
    'INSERT INTO ingredient_consumption
    (production_plan_id, order_id, product_id, consumed_quantity, consumption_source, consumed_by)
    VALUES (:production_plan_id, :order_id, :product_id, :consumed_quantity, :source, :consumed_by)'
);

$stmt->execute([
    'production_plan_id' => $data['production_plan_id'] ?? null,
    'order_id' => $data['order_id'] ?? null,
    'product_id' => $data['product_id'],
    'consumed_quantity' => $data['consumed_quantity'],
    'source' => $data['consumption_source'],
    'consumed_by' => $user['id'],
]);

$stock = $db->prepare(
    'UPDATE stock_items
    SET current_quantity = current_quantity - :quantity
    WHERE product_id = :product_id'
);

$stock->execute([
    'quantity' => $data['consumed_quantity'],
    'product_id' => $data['product_id'],
]);

AuditLogger::log(
    (int) $user['id'],
    'PRODUCTION',
    'RECORD_INGREDIENT_CONSUMPTION',
    (int) $data['product_id'],
    null,
    $data
);

$db->commit();

Response::success([], 'Ingredient consumption recorded');
}

public function recordWastage(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF']);
    $data = Request::body();
    $db = Database::connection();

    $db->beginTransaction();

    $stmt = $db->prepare(
        'INSERT INTO food_wastage
        (product_id, production_plan_id, wasted_quantity, estimated_cost, wastage_reason, declared_by)
        VALUES (:product_id, :production_plan_id, :wasted_quantity, :estimated_cost, :reason, :declared_by)'
    );

    $stmt->execute([
        'product_id' => $data['product_id'],
        'production_plan_id' => $data['production_plan_id'] ?? null,
        'wasted_quantity' => $data['wasted_quantity'],
        'estimated_cost' => $data['estimated_cost'] ?? 0,
        'reason' => $data['wastage_reason'] ?? null,
        'declared_by' => $user['id'],
    ]);

    $stock = $db->prepare(
        'UPDATE stock_items
        SET current_quantity = current_quantity - :quantity
        WHERE product_id = :product_id'
    );

    $stock->execute([
        'quantity' => $data['wasted_quantity'],
        'product_id' => $data['product_id'],
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'PRODUCTION',
        'RECORD_WASTAGE',
        (int) $data['product_id'],
        null,
        $data
    );
};

```

```

    $db->commit();

    Response::success([], 'Food wastage recorded');
}

public function foodCostReport(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            r.id AS recipe_id,
            r.recipe_name,
            p.product_name AS menu_item,
            p.selling_price,
            COALESCE(SUM(ri.quantity_required * ri.unit_cost), 0) AS food_cost_per_plate,
            p.selling_price - COALESCE(SUM(ri.quantity_required * ri.unit_cost), 0) AS gross_margin_per_plate
        FROM recipes r
        JOIN products p ON p.id = r.menu_product_id
        LEFT JOIN recipe_items ri ON ri.recipe_id = r.id
        GROUP BY r.id, r.recipe_name, p.product_name, p.selling_price
        ORDER BY gross_margin_per_plate DESC"
    );

    Response::success(['food_cost_report' => $stmt->fetchAll()]);
}

public function menuEngineeringReport(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'CHEF', 'ACCOUNTANT']);

    $stmt = Database::connection()->query(
        "SELECT
            p.id AS product_id,
            p.product_name,
            p.product_category,
            p.selling_price,
            p.cost_price,
            COALESCE(SUM(oi.quantity), 0) AS quantity_sold,
            COALESCE(SUM(oi.total_price), 0) AS sales_value,
            COALESCE(SUM(oi.quantity * p.cost_price), 0) AS estimated_cost,
            COALESCE(SUM(oi.total_price), 0) - COALESCE(SUM(oi.quantity * p.cost_price), 0) AS gross_profit
        FROM products p
        LEFT JOIN order_items oi ON oi.product_id = p.id
        LEFT JOIN orders o ON o.id = oi.order_id AND o.order_status = 'PAID'
        WHERE p.product_category = 'FOOD'
        GROUP BY p.id, p.product_name, p.product_category, p.selling_price, p.cost_price
        ORDER BY gross_profit DESC, quantity_sold DESC"
    );

    Response::success(['menu_engineering_report' => $stmt->fetchAll()]);
}

```

21H. Update

apps/admin-web/src/features/operations/api/operationsApi.js

```

recipes: async () => {
    const { data } = await apiClient.get('/production/recipes')
    return data.data.recipes
},

createRecipe: async (payload) => {
    const { data } = await apiClient.post('/production/recipes', payload)
    return data
},

productionPlans: async () => {
    const { data } = await apiClient.get('/production/plans')
    return data.data.production_plans
},

createProductionPlan: async (payload) => {
    const { data } = await apiClient.post('/production/plans', payload)
    return data
},

startProductionPlan: async (id) => {
    const { data } = await apiClient.post(`/production/plans/${id}/start`, {})
    return data
},

completeProductionPlan: async (id, payload) => {
    const { data } = await apiClient.post(`/production/plans/${id}/complete`, payload)
}

```

```

    return data
  },

  recordIngredientConsumption: async (payload) => {
    const { data } = await apiClient.post('/production/ingredient-consumption', payload)
    return data
  },

  recordWastage: async (payload) => {
    const { data } = await apiClient.post('/production/wastage', payload)
    return data
  },

  foodCostReport: async () => {
    const { data } = await apiClient.get('/production/food-cost-report')
    return data.data.food_cost_report
  },

  menuEngineeringReport: async () => {
    const { data } = await apiClient.get('/production/menu-engineering-report')
    return data.data.menu_engineering_report
  },

```

211.

apps/admin-web/src/features/production/pages/RecipesPage.jsx

```

import React, { useState } from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import Input from '../../../components/ui/Input'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function RecipesPage() {
  const qc = useQueryClient()
  const [form, setForm] = useState({
    menu_product_id: '',
    recipe_name: '',
    serving_size: '1',
    ingredient_product_id: '',
    quantity_required: '',
    unit_cost: ''
  })

  const { data, isLoading, error } = useQuery({
    queryKey: ['recipes'],
    queryFn: operationsApi.recipes,
  })

  const mutation = useMutation({
    mutationFn: operationsApi.createRecipe,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['recipes'] }),
  })

  function submit(e) {
    e.preventDefault()

    mutation.mutate({
      menu_product_id: Number(form.menu_product_id),
      recipe_name: form.recipe_name,
      serving_size: Number(form.serving_size),
      items: [
        {
          ingredient_product_id: Number(form.ingredient_product_id),
          quantity_required: Number(form.quantity_required),
          unit_cost: Number(form.unit_cost),
        },
      ],
    })
  }

  return (
    <Page title="Recipes / BOM" subtitle="Set food recipe and ingredient bill of materials">
      <Card title="Create Recipe">
        {mutation.error ? <StatusMessage error={mutation.error.message} /> : null}
        <form onSubmit={submit} style={{ display: 'grid', gap: 10, maxWidth: 560 }}>
          <Input placeholder="Menu Product ID" value={form.menu_product_id} onChange={(e) => setForm({ ...form, menu_product_id: e.target.value })} />
          <Input placeholder="Recipe Name" value={form.recipe_name} onChange={(e) => setForm({ ...form,

```

```

recipe_name: e.target.value }}} />
    <Input placeholder="Serving Size" value={form.serving_size} onChange={(e) => setForm({ ...form,
serving_size: e.target.value })} />
    <Input placeholder="Ingredient Product ID" value={form.ingredient_product_id} onChange={(e) =>
setForm({ ...form, ingredient_product_id: e.target.value })} />
    <Input placeholder="Quantity Required" value={form.quantity_required} onChange={(e) => setForm({
...form, quantity_required: e.target.value })} />
    <Input placeholder="Unit Cost" value={form.unit_cost} onChange={(e) => setForm({ ...form, unit_cost:
e.target.value })} />
    <Button type="submit">Create Recipe</Button>
  </form>
</Card>

<Card title="Recipes">
  {isLoading ? <LoadingBlock label="Loading recipes..." /> : null}
  {error ? <StatusMessage error={error.message} /> : null}
  {!isLoading && !error ? (
    <DataTable
      columns={[
        { key: 'recipe_name', label: 'Recipe' },
        { key: 'menu_product_name', label: 'Menu Item' },
        { key: 'serving_size', label: 'Serving Size' },
        { key: 'is_active', label: 'Active' },
      ]}
      rows={data || []}
    />
  ) : null}
</Card>
</Page>
)
}

```

21J.

apps/admin-web/src/features/production/pages/ProductionPlansPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function ProductionPlansPage() {
  return (
    <OperationalFormPage
      title="Kitchen Production Plans"
      subtitle="Plan kitchen production and monitor planned vs produced food"
      queryKey="production-plans"
      queryFn={operationsApi.productionPlans}
      mutationFn={operationsApi.createProductionPlan}
      initialForm={{
        production_date: '',
        menu_product_id: '',
        planned_quantity: '',
      }}
      buildPayload={({form) => ({
        ...form,
        menu_product_id: Number(form.menu_product_id),
        planned_quantity: Number(form.planned_quantity),
      })}
      fields={[
        { name: 'production_date', label: 'Production Date', type: 'date' },
        { name: 'menu_product_id', label: 'Menu Product ID', type: 'number' },
        { name: 'planned_quantity', label: 'Planned Quantity', type: 'number' },
      ]}
      columns={[
        { key: 'production_date', label: 'Date' },
        { key: 'product_name', label: 'Menu Item' },
        { key: 'planned_quantity', label: 'Planned' },
        { key: 'produced_quantity', label: 'Produced' },
        { key: 'sold_quantity', label: 'Sold' },
        { key: 'wasted_quantity', label: 'Wasted' },
        { key: 'production_status', label: 'Status' },
      ]}
      submitLabel="Create Production Plan"
    />
  )
}

```

21K.

apps/admin-web/src/features/production/pages/IngredientConsumptionPage.jsx

```

import React from 'react'

```



```

import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function IngredientConsumptionPage() {
  return (
    <OperationalFormPage
      title="Ingredient Consumption"
      subtitle="Record ingredients consumed by production plan or order"
      queryKey="stock-movement-report"
      queryFn={operationsApi.stockMovementReport}
      mutationFn={operationsApi.recordIngredientConsumption}
      initialForm={{
        production_plan_id: '',
        order_id: '',
        product_id: '',
        consumed_quantity: '',
        consumption_source: 'PRODUCTION_PLAN',
      }}
      buildPayload={({form}) => ({
        production_plan_id: form.production_plan_id ? Number(form.production_plan_id) : null,
        order_id: form.order_id ? Number(form.order_id) : null,
        product_id: Number(form.product_id),
        consumed_quantity: Number(form.consumed_quantity),
        consumption_source: form.consumption_source,
      })
      fields={[
        { name: 'production_plan_id', label: 'Production Plan ID optional', type: 'number' },
        { name: 'order_id', label: 'Order ID optional', type: 'number' },
        { name: 'product_id', label: 'Ingredient Product ID', type: 'number' },
        { name: 'consumed_quantity', label: 'Consumed Quantity', type: 'number' },
        {
          name: 'consumption_source',
          label: 'Consumption Source',
          type: 'select',
          options: [
            { value: 'PRODUCTION_PLAN', label: 'Production Plan' },
            { value: 'ORDER', label: 'Order' },
            { value: 'MANUAL_ADJUSTMENT', label: 'Manual Adjustment' },
          ],
        },
      ]}
      columns={[
        { key: 'product_name', label: 'Product' },
        { key: 'movement_type', label: 'Movement' },
        { key: 'quantity', label: 'Quantity' },
        { key: 'reference_number', label: 'Reference' },
        { key: 'created_at', label: 'Date' },
      ]}
      submitLabel="Record Consumption"
    />
  )
}

```

21L.

apps/admin-web/src/features/production/pages/WastageControlPage.jsx

```

import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function WastageControlPage() {
  return (
    <OperationalFormPage
      title="Wastage Control"
      subtitle="Record food wastage and reduce production loss"
      queryKey="ops-unsold-food"
      queryFn={operationsApi.unsoldFood}
      mutationFn={operationsApi.recordWastage}
      initialForm={{
        product_id: '',
        production_plan_id: '',
        wasted_quantity: '',
        estimated_cost: '',
        wastage_reason: '',
      }}
      buildPayload={({form}) => ({
        product_id: Number(form.product_id),
        production_plan_id: form.production_plan_id ? Number(form.production_plan_id) : null,
        wasted_quantity: Number(form.wasted_quantity),
        estimated_cost: Number(form.estimated_cost || 0),
        wastage_reason: form.wastage_reason,
      })
      fields=[

```

```

    { name: 'product_id', label: 'Product ID', type: 'number' },
    { name: 'production_plan_id', label: 'Production Plan ID optional', type: 'number' },
    { name: 'wasted_quantity', label: 'Wasted Quantity', type: 'number' },
    { name: 'estimated_cost', label: 'Estimated Cost', type: 'number' },
    { name: 'wastage_reason', label: 'Wastage Reason', type: 'textarea' },
  ]
}
columns=[
  { key: 'product_name', label: 'Product' },
  { key: 'quantity', label: 'Quantity' },
  { key: 'reason', label: 'Reason' },
  { key: 'declared_by_name', label: 'Declared By' },
  { key: 'created_at', label: 'Date' },
]
submitLabel="Record Wastage"
/>
)
}

```

21M.

apps/admin-web/src/features/production/pages/MenuEngineeringReportPage.jsx

```

import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function MenuEngineeringReportPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['menu-engineering-report'],
    queryFn: operationsApi.menuEngineeringReport,
  })

  return (
    <Page title="Menu Engineering Report" subtitle="Sales, cost, and gross profit per food menu item">
      <Card>
        {isLoading ? <LoadingBlock label="Loading menu engineering report..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}

        {!isLoading && !error ? (
          <DataTable
            columns=[
              { key: 'product_name', label: 'Menu Item' },
              { key: 'quantity_sold', label: 'Qty Sold' },
              { key: 'sales_value', label: 'Sales' },
              { key: 'estimated_cost', label: 'Cost' },
              { key: 'gross_profit', label: 'Gross Profit' },
            ]
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}

```

21N.

apps/admin-web/src/features/production/pages/FoodCostReportPage.jsx

```

import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../../operations/api/operationsApi'

export default function FoodCostReportPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['food-cost-report'],
    queryFn: operationsApi.foodCostReport,
  })

  return (
    <Page title="Food Cost Report" subtitle="Food cost per plate and gross margin per menu item">
      <Card>

```

```

{isLoading ? <LoadingBlock label="Loading food cost report..." /> : null}
{error ? <StatusMessage error={error.message} /> : null}

{!isLoading && !error ? (
  <DataTable
    columns={[
      { key: 'recipe_name', label: 'Recipe' },
      { key: 'menu_item', label: 'Menu Item' },
      { key: 'selling_price', label: 'Selling Price' },
      { key: 'food_cost_per_plate', label: 'Food Cost' },
      { key: 'gross_margin_per_plate', label: 'Margin' },
    ]}
    rows={data || []}
  />
) : null}
</Card>
</Page>
)
}

```

21O. Update

apps/admin-web/src/app/routes.jsx

```

import RecipesPage from '../features/production/pages/RecipesPage'
import ProductionPlansPage from '../features/production/pages/ProductionPlansPage'
import IngredientConsumptionPage from '../features/production/pages/IngredientConsumptionPage'
import WastageControlPage from '../features/production/pages/WastageControlPage'
import MenuEngineeringReportPage from '../features/production/pages/MenuEngineeringReportPage'
import FoodCostReportPage from '../features/production/pages/FoodCostReportPage'

```

Add routes:

```

<Route path="/production/recipes" element={<RecipesPage />} />
<Route path="/production/plans" element={<ProductionPlansPage />} />
<Route path="/production/ingredient-consumption" element={<IngredientConsumptionPage />} />
<Route path="/production/wastage" element={<WastageControlPage />} />
<Route path="/production/menu-engineering" element={<MenuEngineeringReportPage />} />
<Route path="/production/food-cost" element={<FoodCostReportPage />} />

```

21P. Update

apps/admin-web/src/components/common/SidebarNav.jsx

```

['Recipes / BOM', '/production/recipes'],
['Production Plans', '/production/plans'],
['Ingredient Consumption', '/production/ingredient-consumption'],
['Wastage Control', '/production/wastage'],
['Menu Engineering', '/production/menu-engineering'],
['Food Cost Report', '/production/food-cost'],

```

21Q.

docs/PRODUCTION_MENU_ENGINEERING_PACK.md

Batch 21 – Production + Menu Engineering Pack

System

Meat Lovers CIMS powered by YohPal

Purpose

This batch controls kitchen production, recipes, food cost, and menu profitability.

Features Added

1. Recipe/BOM setup
2. Recipe ingredients
3. Kitchen production plans
4. Ingredient consumption
5. Food wastage control
6. Food cost per plate
7. Gross margin per menu item
8. Menu engineering report

Recipe/BOM

Each food menu item can have a recipe defining:

- ingredient product
- quantity required
- unit cost

Kitchen Production Plans

Production plans track:

- planned quantity
- produced quantity
- sold quantity
- wasted quantity
- production status

Ingredient Consumption

Ingredient consumption reduces stock quantity.

Sources:

- production plan
- order
- manual adjustment

Wastage Control

Wastage records:

- product
- quantity wasted
- estimated cost
- reason
- person declaring wastage

Menu Engineering

The system calculates:

- quantity sold
- sales value
- estimated cost
- gross profit

Business Value

This helps Meat Lovers:

- know food cost per plate
- identify profitable food items
- reduce kitchen wastage
- control ingredient usage
- stop kitchen stock leakage
- improve menu pricing decisions

Batch 21 Outcome

Production + menu engineering now includes:

- ✅ recipe/BOM setup
- ✅ kitchen production plans
- ✅ ingredient consumption
- ✅ food cost per plate
- ✅ gross margin per menu item
- ✅ wastage control
- ✅ menu engineering report

Next Smart Move

Build **Batch 22 — Product Segmentation + Pricing Control Pack**, including category-specific pricing rules, food/soft drink/alcohol dashboards, margin alerts, inactive product control, and price-change audit trail.